

Complete Setup Guide

This guide reproduces the Gmail Job Tracker from an empty Google Sheet and a standalone Google Apps Script project.

1. Prerequisites

You need:

- A Google account with Gmail
- Google Sheets
- Google Apps Script access
- A Gmail account that receives job application and recruiter emails

Use a test account or synthetic messages during initial testing when possible.

2. Create the Google Sheet

Create a new Google Sheet and rename the first tab to:

TEMPLATE

Build two sections.

Recruiter / Vendor section

Merge A1:H1 and enter:

RECRUITER / VENDOR EMAILS SENT

In row 2 enter:

A	B	C	D	E	F	G	H
Date	Company	Recruiter Name	Phone Number	Email	Job Title	Status	Email Link

Leave rows 3-6 blank.

Portal / Direct Application section

Merge A7:G7 and enter:

PORTAL / DIRECT JOB APPLICATIONS

In row 8 enter:

A	B	C	D	E	F	G
Date	Portal	Company	Position	Application #	Status	Email Link

The automation preserves four blank rows between the recruiter records and the portal section.

3. Copy the spreadsheet ID

From the Sheet URL:

https://docs.google.com/spreadsheets/d/SPREADSHEET_ID/edit

copy only the SPREADSHEET_ID portion.

4. Create the Apps Script project

Create a standalone Apps Script project and give it a recognizable name such as:

Job Tracker Automation

The repository keeps the Apps Script source in the **src/** directory. Add every **.gs** file from **src/** to the same Apps Script project. Apps Script loads project files together, so the functions can call one another across files.

5. Configure the Sheet ID

The public repository intentionally contains:

```
const SPREADSHEET_ID = 'YOUR_GOOGLE_SHEET_ID_HERE';
```

Replace that placeholder in your private Apps Script project with your Sheet ID.

Do not commit the real ID back to a public repository.

6. Save and test the Sheet connection

Run:

testSpreadsheetConnection

Authorize the requested Google permissions.

A successful execution should log that the spreadsheet was connected and list the available sheets.

7. Test daily sheet creation

Run:

createTodaySheet

A new sheet named YYYY-MM-DD should be copied from **TEMPLATE**.

Running the function again on the same day should reuse the existing daily sheet instead of creating a duplicate.

8. Test recruiter/vendor detection

The project includes:

testRecruiterSentEmails

Use synthetic recruiter-related sent emails while testing. The detector looks for job/recruiting language and can also use earlier messages in the same Gmail thread as context.

9. Test recruiter writing

Run:

syncRecruiterSentEmailsToSheet

Expected behavior:

- Today's recruiter/vendor sent messages are detected.
- A structured row is inserted into today's sheet.
- The Portal section moves down automatically.
- Exactly four blank rows remain between sections.
- The Gmail message is recorded in **_Processed** for duplicate prevention.

10. Test portal/direct applications

Use:

```
testPortalApplicationEmails
```

then:

```
syncPortalApplicationsToSheet
```

The application detector uses confirmation phrases, sender/ATS signals, and application language. Job alerts and generic recommendation messages receive negative weighting unless there is strong application-confirmation evidence.

11. Supported portal identification

The code contains sender-based mappings for common platforms including:

- Monster
- LinkedIn
- Dice
- Indeed
- ZipRecruiter
- Workday
- Greenhouse
- Lever
- iCIMS
- SmartRecruiters
- Jobvite
- Taleo
- Oracle Recruiting
- SuccessFactors
- Ashby
- Workable

Unknown direct employer senders can be classified as `Company Portal`.

12. Configure recruiter status dropdowns

Run:

```
updateRecruiterStatusDropdowns
```

Recruiter statuses are:

```
Sent  
Follow-up Sent  
Replied  
RTR  
Submitted  
Assessment  
Interview  
Rejected  
Offer  
Closed
```

13. Test recruiter status updates

Run:

```
syncRecruiterStatuses
```

The matcher uses Gmail thread context and additional recruiter/job/company evidence to reduce false matches.

14. Test application status updates

Run:

```
syncApplicationStatuses
```

Portal statuses are:

```
Applied
Assessment
Interview
Rejected
Offer
Withdrawn
```

Promotional/community/job-alert language is intentionally filtered so generic messages are less likely to be treated as status updates.

15. Test the full normal automation

Run:

```
runJobTrackerAutomation
```

The normal workflow performs:

1. Recruiter/vendor sent sync
2. Recruiter/vendor status sync
3. Portal/direct application sync
4. Application status sync
5. Dashboard refresh

LockService is used to reduce overlapping master runs.

16. Create All Jobs

Run:

```
createAllJobsSheet
```

This creates a permanent master sheet based on the tracker template and applies the status dropdowns.

17. Historical recruiter backfill

Run only when you intentionally want to import historical Gmail data:

```
startAllJobsRecruiterBackfill
```

The function creates a temporary trigger and processes Gmail in batches. Do not repeatedly restart the start function during an active backfill because the saved position is initialized by the starter.

Check progress with:

```
checkAllJobsBackfillProgress
```

18. Historical portal application backfill

After recruiter history is complete, run:

```
startAllJobsPortalBackfill
```

19. Historical portal status backfill

After portal application history is complete, run:

```
startAllJobsPortalStatusBackfill
```

20. Historical recruiter status backfill

After the earlier historical stages are complete, run:

```
startAllJobsRecruiterStatusBackfill
```

The project also contains emergency stop functions for each historical workflow.

21. Dashboard

Run:

```
refreshJobDashboard
```

The Dashboard reports current counts for recruiter/vendor activity and portal applications, including status breakdowns.

Because the Dashboard reads `All Jobs`, historical counts will continue changing while a backfill is still running.

22. Create the live trigger

In Apps Script:

1. Open **Triggers**.
2. Click **Add Trigger**.
3. Select `runJobTrackerAutomation`.
4. Event source: **Time-driven**.
5. Choose an hourly timer for a conservative default.
6. Save.

Apps Script time-driven triggers are scheduled, not hard real-time guarantees.

23. Verification checklist

Confirm all of the following:

- `TEMPLATE` exists.
- Today's `YYYY-MM-DD` sheet is created automatically.
- Recruiter emails appear only once.
- Portal applications appear only once.
- Gmail links open the expected thread.
- Recruiter status progression behaves correctly.
- Portal status progression behaves correctly.
- `All Jobs` exists.
- `_Processed` is hidden.
- Dashboard refresh succeeds.
- Hourly trigger appears on the Triggers page.

24. Privacy checklist before screenshots or GitHub publication

Remove or replace:

- Real recruiter names
- Email addresses
- Phone numbers
- Application numbers
- Gmail thread IDs
- Google Sheet IDs
- Personal Gmail addresses
- Any company/job details you do not want public

Use synthetic records for the public demo.

25. Maintenance

When changing keyword or status rules:

1. Test with synthetic messages first.
2. Review false positives and false negatives.
3. Verify duplicate processing is unchanged.
4. Verify status progression cannot move backward unexpectedly.
5. Update this repository's documentation and changelog.